LoRa-Based Encryption and Communications Testbed

Homar A. Rodriguez | hrodri16@jh.edu & **Angel K. Vazquez** | avazqu17@jh.edu

Prof. Jeff Houser

Abstract

This project develops a two-node laboratory prototype for secure LoRa communications with host-controlled experiment execution and measurement. Each embedded node transmits and receives packetized data over a configurable 915 MHz LoRa link, applies encryption and authentication to protect message contents, and reports telemetry to a host-side testbed over USB serial; after initial data-link validation, the system will also support compressed voice transport within the practical bandwidth limits of LoRa. The primary objective is to both demonstrate secure wireless communication and create a repeatable experimental platform that compares how radio settings, packet formatting, and security mechanisms affect packet delivery, payload throughput/delay, with the final system emphasizing controlled testing, logging, and analysis rather than field-ready deployment.

1. Project Description

1.1 High-Level Description

The LoRa-Based Encryption and Communications Testbed is a two-node embedded communications platform designed for controlled laboratory evaluation of secure low-rate wireless links. The system consists of two ESP32 microcontrollers with a LoRa transceiver, and a testbed application that configures experiments, coordinates execution, logs telemetry, and analyzes performance.

Each node supports transmission and reception operations. On transmission, the node accepts input data, encodes voice messages, fragments the message into packets, appends sequencing metadata, applies encryption and authentication, and passes the protected payload to the LoRa radio driver for broadcast. On reception, the node accepts packets from the radio, records signal telemetry such as RSSI and SNR, verifies integrity, checks for replay or duplication, decrypts valid packets, reassembles complete messages, and outputs recovered data or decoded audio.

The testbed forms the measurement and orchestration layer of the project. It communicates with both nodes through USB serial links, applies experiment settings, starts and stops test runs, collects node telemetry and error events, stores run records, and generates visualizations for comparison across different radio and security configurations. This architecture allows the project to function both as a secure communications prototype and as a repeatable experimental platform.

1.2 Fit Within a Larger System

This project is a reduced-scope prototype of a larger secure wireless communication system. In a broader deployment, the same architectural approach could support multiple nodes, portable operation, field power, autonomous configuration, longer-duration logging, or integration into a secure sensing or tactical telemetry network. The present course project intentionally limits the scope to two nodes in a controlled bench environment to reduce risk while still demonstrating the essential communication, security, and measurement functions.

The testbed architecture is also intended to be reusable beyond this class. Once the control protocol, telemetry format, and measurement workflow are defined, the host software can be reused to evaluate future node firmware revisions, different LoRa parameter combinations, alternate encryption schemes, or additional application payloads. As a result, the project is not only a prototype communication link but also a reusable framework for development and validation.

1.3 Capabilities

The planned system capabilities are as follows:

- Support a two-way packet exchange between two embedded nodes over a 915 MHz LoRa radio link.
- Record, buffer, and segment a three-second local voice message (resulting in a 48 KB uncompressed buffer at 8 kHz, 16-bit Mono) from an I2S microphone into sequentially indexed packets for point-to-point transmission.
- Configure LoRa radio parameters, including frequency, bandwidth, spreading factor, coding rate, and transmit power.
- Apply packet protection using encryption and message authentication.
- Include sequence and message-position metadata to support packet reassembly and replay detection.
- Collect radio telemetry such as received signal strength and signal-to-noise ratio during test runs.
- Support host-controlled experiments through USB serial connections to both nodes.
- Log experiment settings, timestamps, node identifiers, sequence numbers, and errors for each run.
- Evaluate packet delivery, payload throughput, and round-trip delay across controlled scenarios.
- Demonstrate compressed voice transport over the LoRa radio link.
- Provide a repeatable test environment for comparing communication performance under different radio and security settings.

1.4 Limitations

This project is intentionally constrained to remain feasible within the course schedule and hardware resources.

- The prototype is limited to two nodes and point-to-point operation.
- Initial operation assumes a controlled laboratory environment with nearby nodes and host-connected serial control.
- LoRa bandwidth is limited, so compressed voice support is a feasibility demonstration rather than a guarantee of continuous high-quality audio.
- Delay and throughput depend strongly on spreading factor, bandwidth, coding rate, and packet overhead [6]–[8].
- Initial latency measurement will use round-trip timing because synchronized clocks are not assumed.
- The project emphasizes host-supervised experiments rather than independent or autonomous field operation.
- Power management is included as a support function, but battery optimization is not a primary design objective.
- Security evaluation is limited to the implemented packet protection and replay/integrity mechanisms; it is not a full formal cryptographic certification effort.
- The chunking mechanism requires the Heltec V4's ESP32-S3 to hold the entire voice recording in its 512 KB internal SRAM buffer before slicing and transmitting.
- The chunking mechanism is limited by LoRa's strict maximum payload of 255 bytes per packet, which requires the 48 KB voice recording to be fragmented into over 200 sequentially transmitted packets, risking significant latency and packet loss during reassembly.

2. System Architecture

2.1 High-Level System Architecture

The complete system includes two embedded test nodes and one host PC acting as the testbed control station. Node A and Node B exchange payloads over a LoRa wireless channel in the 915 MHz ISM band. Each node is also connected to the host testbed through USB serial, allowing the host to configure radio settings, load test scenarios, start and stop traffic generation, and collect measurement data.

At the highest level, the system has two major paths:

1. Control and measurement path: The host PC sends commands to the nodes over USB serial and receives replies, telemetry, and event records.
2. Communications path: The two nodes exchange encrypted and authenticated packets over the LoRa radio link.

This separation allows the RF link to serve as the experimental subject, while the wired serial path remains in the reliable control and data-collection channel.

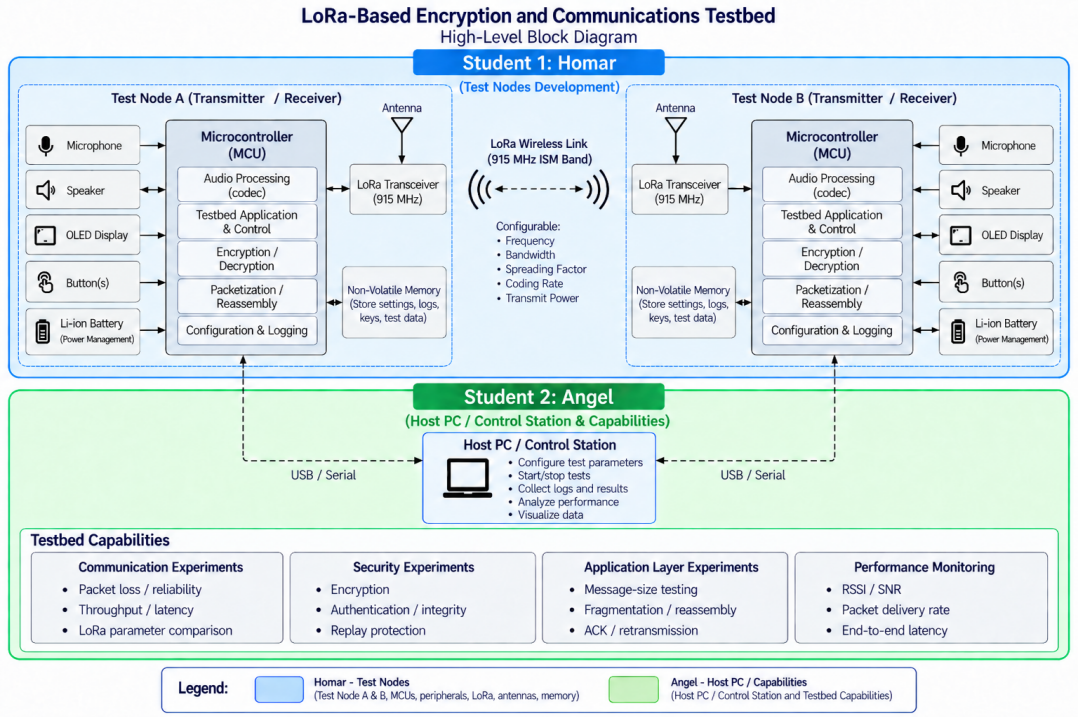


Figure 1. High-Level System Architecture

2.2 Test Node Functional Architecture

Transmission Path

The node accepts either test data or audio input. If voice mode is enabled, the node encodes the captured audio into a compressed representation compatible with the available LoRa throughput

(for example, Codec 2 at 700–3200 bit/s [13]). The resulting data stream is divided into packets (fragmentation schemes for constrained low-power wide-area links are described in [12]), each labeled with sequence and message-position metadata, enabling the receiving node to detect missing fragments and reconstruct complete messages. The packet is then protected with encryption and authentication (for example, an authenticated-encryption mode such as GCM [10]) before being transmitted through the LoRa driver and antenna.

Reception Path

On reception, the node obtains packets and radio signal measurements from the LoRa transceiver. It verifies packet authenticity and integrity, applies replay checks using sequence metadata or related freshness information (for example, a sliding receive window as used in IPsec ESP [11]), decrypts valid packets, and then reassembles them into complete messages. If the message is an encoded audio payload, the node decodes it for output. Otherwise, the recovered data is delivered as application data or test payload output.

Supporting Functions

Each node also includes:

- USB serial communication for testbed control and telemetry
- local controls such as display and buttons where available
- settings storage for persistent configuration
- power management and reset handling
- internal event logging for diagnostics

The node architecture is intentionally modular so that packet formatting, security, radio control, and application payload handling can be tested independently before full integration.

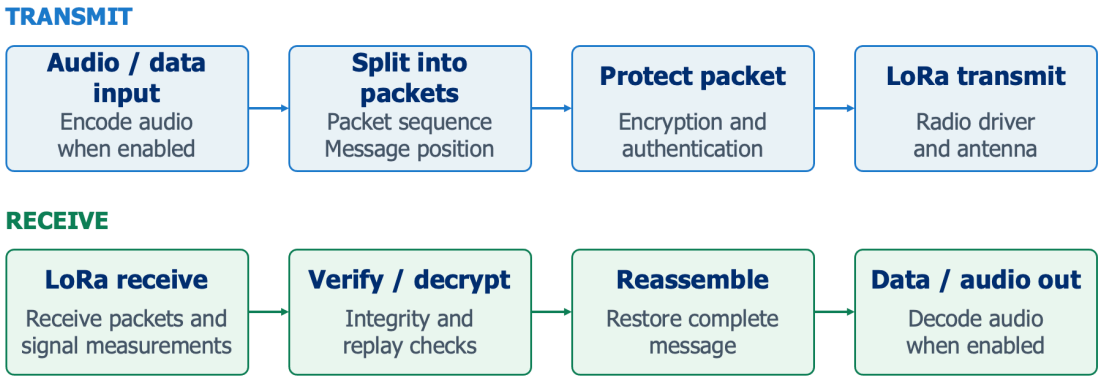


Figure 2. Test Node Functional Architecture

2.3 Testbed Functional Architecture

The testbed application runs on a host PC and serves as the system controller, experiment runner, and results collector.

Control Path

The operator defines scenario settings such as LoRa parameters, packet size, test duration, traffic pattern, retry behavior, and security enable/disable settings. A test runner then executes the scenario by sending commands to Node A and Node B over serial connections. The serial transport layer manages framing, replies, timeouts, and command acknowledgments.

Results Path

During and after each run, the testbed collects telemetry from the nodes. This includes node identifiers, packet counts, timestamps, sequence numbers, RSSI, SNR, success/failure indicators, and error flags. An event collector validates the incoming records and associates them with the active test run. The results are stored as run records that preserve both configuration and observed outcomes. The analysis layer then computes metrics and generates tables or plots to compare runs across configurations.

The host-side architecture is especially important in this project because the main value of the prototype is not simply that packets can be sent, but that performance can be measured repeatably and compared systematically.

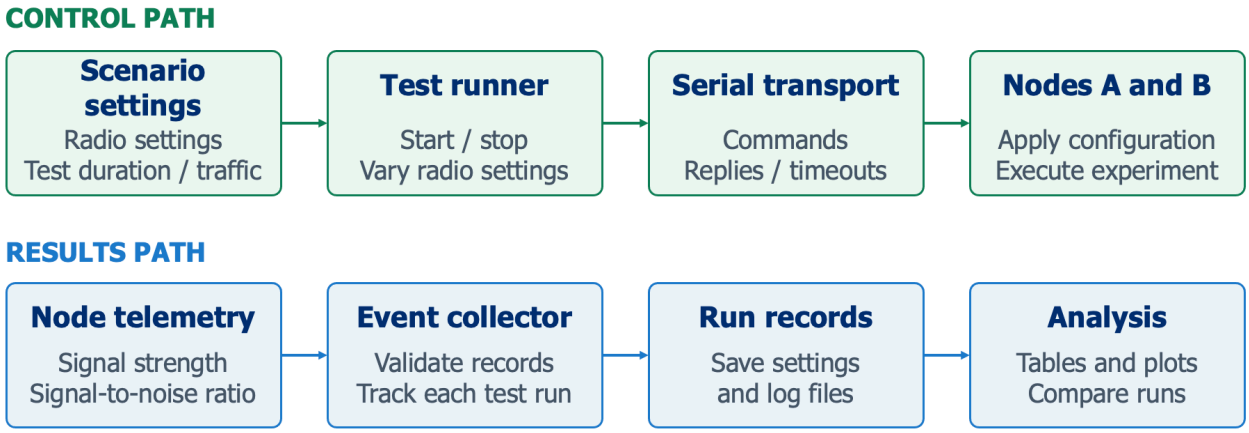


Figure 3. Testbed Functional Architecture

3. Interface Descriptions

3.1 Internal Interfaces

The project includes several internal interfaces within each node and between the nodes and testbed.

Microcontroller to LoRa Radio

The radio transceiver is controlled over an SPI interface integrated into the microcontroller design. This interface is responsible for register configuration, packet loading and unloading, interrupt/status handling, and radio state transitions. It forms the primary internal boundary between firmware and RF hardware.

Microcontroller to Audio Path

Where audio is enabled, a digital audio interface such as I2S carries audio samples and control information between the microcontroller and audio codec or microphone/speaker peripherals. This interface supports audio capture and playback but is secondary to baseline data-link operation.

Microcontroller to Local Controls

Digital GPIO and serial buses support display, buttons, and nonvolatile settings storage. These functions are not central to the communication experiment but support usability, local visibility, and configuration persistence.

Node Firmware Module Interfaces

Within the firmware, internal software interfaces separate:

- application input/output handling
- packetization and reassembly
- encryption/authentication processing
- replay/integrity checking
- LoRa radio driver control
- telemetry and serial command handling

These interfaces will be implemented with clearly defined data structures, so modules can be tested individually.

Testbed to Each Node

Each node connects to the host PC over USB serial. This link carries configuration commands, experiment controls, status messages, and measurement telemetry. Although physically external to the node, it is treated as part of the internal project architecture because both ends are under project control.

3.2 External Interfaces

Node A to Node B LoRa RF Link

The RF interface between the two nodes is the main communication channel under test. It operates in the 915 MHz band using configurable LoRa parameters. This interface is external in the sense that it traverses the radio environment, making it sensitive to placement, interference, and configuration.

User and Bench Interfaces

The system may expose audio connectors, USB power, bench power, or local interaction interfaces such as buttons and display. These are external user-facing interfaces and must be robust enough for repeatable bench testing.

Development and Measurement Environment

The host PC, USB serial ports, and bench power supplies constitute the external setup required to operate and evaluate the system. While not part of the LoRa link itself, these interfaces are necessary to run the project as a controlled testbed.

3.3 Interface Control Considerations

Before integration begins, the team must freeze the interface definitions shared between the node firmware and the host testbed. At minimum, the following items must be defined:

- serial message framing
- command and response formats
- field names and units
- telemetry record structure
- sequence numbering rules
- error codes
- timeout and retry behavior
- protocol version field
- security state indicators
- packet header format
- replay-detection metadata rules

These definitions are critical because the project uses parallel development by two team members. If interfaces are not fixed early, integration risk increases substantially. A preliminary interface summary is shown below.

Connection	Proposed interface	Purpose / boundary
Microcontroller ↔ radio	Serial Peripheral Interface (Internally Integrated)	Radio settings and packets; internal
Microcontroller ↔ audio	Digital audio bus - I2S	Audio samples and control; internal
Microcontroller ↔ controls	Digital pins / serial buses	Display, buttons, storage; internal
Testbed ↔ Each node	USB serial connection	Control and measurements; internal
Node A ↔ Node B	LoRa radio, 915 MHz	Internal protocol / external radio channel
Users / bench ↔ System	Audio and power connectors	Audio and power; external

4. Materials and Resource Requirements



Figure 4. Prototyping workbench for project

4.1 Hardware

The project requires the following hardware resources:

- five ESP32-based boards with LoRa capability, radios, and antennas
- audio codec hardware, microphones, and speakers for voice testing
- display, buttons, batteries, cables, and other integration accessories
- a host computer for the testbed software
- stable power supplies for bench operation
- soldering tools and assembly materials
- logic analyzer and multimeter for debug and interface validation

The use of five boards instead of exactly two allows flexibility for replacement, spare units, and subsystem testing without disrupting the main integration hardware.

4.2 Software and Development Tools

The primary development tools are expected to include:

- Espressif ESP-IDF toolchain
- C/C++ firmware development tools

- Git for revision control
- Serial terminal utilities
- Python for host-side testbed automation and analysis
- Data analysis and plotting libraries as needed
- Board support packages and radio driver libraries compatible with the chosen hardware

If third-party libraries are used for encryption, audio processing, or radio abstraction, their licenses and integration constraints must be documented in the final report.

4.3 Source Availability and Procurement Status

At the time of CDR, the major project hardware is identified as team-owned or team-stocked, reducing procurement risk. The current known material status is summarized below:

Item	Quantity	Source / status	Needed by
Heltec V4 ESP32 Kit	5	Team stock / Owned	Week 1-2
MAX98357A I2S	12	Team stock / Owned	Week 1-2
1 Watt 8 Ohm Cavity Mini Speaker 3.3V – 5.0V Full Range Small Speakers with JST-PH 1.25mm	12	Team stock / Owned	Weeks 1–2
INMP441 I2S Microphone Module	9	Open Source	Week 2
180Pcs Tactile Push Button Switch Assortment Kit	1	Team stock / Owned	Week 1
Logic analyzer, Multimeter	Logic analyzer, Multimeter	Team stock / Owned	Weeks 2–4
Two Soldering Station	Two Soldering Station	Team stock / Owned	Week 1

Because the key components are already available, schedule exposure from procurement is relatively low. The remaining material risks relate more to hardware integration and compatibility than to delivery time.

4.4 Test Equipment

The following equipment is required to validate the design:

- Host PC for experiment control
- Serial consoles for node bring-up and logging
- Logic analyzer for SPI, UART, and timing verification
- Multimeter for power-path verification and fault diagnosis
- Stable bench power to reduce resets and inconsistent measurements
- Oscilloscope for deeper timing or power analysis

4.5 Required Knowledge Base

Successful completion of the project requires working knowledge in:

- Embedded C/C++ development
- ESP32 platform and ESP-IDF build flow
- LoRa radio configuration and tradeoffs
- Serial protocol design
- Authenticated encryption concepts
- Packet sequencing and replay protection
- Test automation and experiment logging
- Measurement analysis and plotting
- Audio sampling and compression tradeoffs for constrained links

5. Development Plan and Schedule

5.1 Development Approach

The project will use an incremental integration strategy. The priority is to establish a reliable baseline communication path for known test data before introducing voice functionality. Encryption, telemetry, and host-driven experiment control will be added in stages so that failures can be isolated to a single interface or subsystem.

The design approach intentionally separates the project into two main areas of ownership:

- Embedded test node implementation
- Host testbed implementation

This allows parallel development while keeping the major integration boundary limited to a defined serial control and telemetry interface.

5.2 Order of Development

The planned order of implementation is:

1. Assemble and verify baseline node hardware and software environment.
2. Bring up serial communications between each node and the host.
3. Implement baseline LoRa packet transmit/receive using known payload data.
4. Define and freeze packet and telemetry formats.
5. Add encryption/decryption and message authentication.
6. Implement packet sequencing, fragmentation, and reassembly.
7. Add replay detection and negative test cases.
8. Build host-side scenario configuration, logging, and experiment automation.
9. Integrate the host testbed with two active nodes.
10. Execute controlled radio-parameter experiments and collect performance data.
11. Add compressed voice path if link-budget and throughput measurements support it.
12. Perform final system validation and demonstration preparation.

This order front-loads the highest-value and highest-risk features first: node communication, interface definition, and security integration.

5.3 Responsibilities and Dependencies

The project responsibilities are divided as follows:

Homar - Test Nodes

- Hardware and peripheral integration
- LoRa firmware and packet handling

- Node authentication
- Node encryption and decryption
- Audio processing and local controls

Angel - Testbed

- Serial command and telemetry handling
- Test configuration and automation
- Data collection and run records
- Performance analysis and plots
- Final experiment procedure definition

Shared Responsibility

- Interface definitions
- Packet format
- Command vocabulary
- Telemetry fields
- Timing assumptions
- Integration checkpoints

The major dependency is that the node firmware and the host testbed must agree on a stable command-and-telemetry contract before parallel development proceeds too far.

5.4 Integration and Test Plan

Integration will proceed in three stages:

Stage 1: Interface Test

The host sends known commands to each node and verifies valid replies, error handling, and telemetry formatting. This stage checks USB serial reliability, framing, and timeout behavior without dependence on the RF link.

Stage 2: Integrated Link

Node A and Node B exchange known data over LoRa while the host monitors command and telemetry flow. This stage confirms packet handling, sequencing, and basic RF operation.

Stage 3: Experiment Run

The host configures complete scenarios, starts tests, collects data automatically, and stores results for analysis. This stage validates the system as a true testbed rather than a manually operated demo.

These stages map directly to the architecture and reduce the chance of discovering multiple independent failures at final integration.

5.5 Milestones and Schedule

The current planned schedule is summarized below:

When	Milestone (Test Nodes)	Milestone (Testbed)
Week 1-2	Software Setup / Nodes Assembly	Software Setup / Experiment Design
Week 3	Sensors Integration / Data Encryption	Logging and Telemetry / Data Processing
Week 4-5	Node Integration Testing/Troubleshooting	Host-2-Node Control & Data Interfaces
Week 6-8	Testbed/Node integration	
Week 9-10	Performance Testing	
Week 10	Final Demonstration	

6. Risks and Mitigations

The principal project risks are listed below:

Risk	Impact	Mitigation
Limited LoRa bandwidth for voice	Packet loss, distorted audio	Reduce audio bitrate / compression
End-to-End delay	Audio size can cause transmission delays	Short and controlled audio frames
Memory limitations	Audio buffers share memory resources with microcontroller systems	Memory use measurements and buffer size limits
Testbed and node timing mismatch	Inaccurate latency measurements	Roundtrip measurements
Audio codec integration problems	Microphone or speaker path fails	Test audio capture and playback separately before integrating with LoRa
Serial communication failures	Missing commands or incomplete experiment logs	Message framings, acknowledgements and timeouts
Encryption error	Nodes cannot communicate or message replay fails	Key loading and test known messages
RF performance varies with environment	Testing not accounting environment conditions	Controlled environment
Power Instability	Unexpected reset or inconsistent measurements	Use stable bench power during initial testing and log reset causes

The most significant technical risk is the mismatch between LoRa's low data rate and the overhead introduced by packetization, encryption, acknowledgments, and possible voice transport [8]. For that reason, the baseline success criterion is secure data communication with repeatable measurement, while voice remains an extension objective subject to measured feasibility.

7. Performance Measurement Plan

The project will evaluate the system through structured experiments. The major measurement categories are radio performance, security behavior, packet handling, and voice feasibility.

7.1 Metrics

Packet Delivery

Packet delivery ratio (PDR) will be defined as:

$$packet\ delivery\ ratio\ =\ \frac{unique\ packets\ received}{packets\ sent}$$

This metric excludes duplicates and focuses on whether intended packets arrive successfully. Its complement (1 − PDR) corresponds to the packet loss ratio defined in the IETF IPPM one-way loss metric [15].

Payload Throughput

Payload throughput will be defined as:

$$payload\ throughput\ =\ \frac{delivered\ payload\ bits}{elapsed\ time}$$

This metric reflects useful application data rather than raw radio airtime.

Delay

Round-trip delay will be measured initially without requiring synchronized clocks. One-way delay measurements will be incorporated in later testing using time synchronization between nodes. Round-trip measurement follows the IETF IPPM round-trip delay metric [14].

Security Behavior

Security-related validation, informed by documented replay and bit-flipping attacks against LoRaWAN [9], will include:

- Successful encrypted communication between legitimate nodes
- Rejection of altered packets
- Rejection of replayed packets
- Measurement of encryption/decryption processing overhead

Packet Handling & Memory Constraint

Packetization testing will evaluate complete message reconstruction under fragmentation, the effect of message size on delivery ratio, and retry/acknowledgment behaviors. To measure memory footprint constraints, the system follows the metrics established in this reference site [16] for voice message payload size and chunking size constraints:

Voice Message Payload Size (S_{voice})

The baseline byte size of an uncompressed audio capture buffer, defined by:

$$S_{\text{voice}} = R_{\text{sample}} \times \left(\frac{B_{\text{depth}}}{8}\right) \times T_{\text{clip}}$$

Where *Rsample* is the I2S sample rate (Hz), *Bdepth* is the bit depth (bits), and T_{clip} is the fixed duration of three seconds. For this project's default testing profile (*Rsample = 8000 HZ, Bdepth = 16 bits,* T_{clip} *= 3 seconds*), the baseline voice payload size is fixed at exactly 48,000 bytes (48 KB).

SRAM/PSRAM Chunking Size Constraint (N_{chunks})

The allocation footprint required by the ESP32-S3 memory manager to slice the monolithic buffer into independent MTU-compliant radio blocks:

$$N_{\text{chunks}} = \left\lceil \frac{S_{\text{voice}}}{P_{\max} - H_{\text{size}}} \right\rceil$$

Where P_{max} is the physical LoRa maximum payload size (255 bytes), and H_{size} is the local framing metadata header size.

Given the 48,000 bytes baseline audio footprint, an optimized maximum payload boundary of P_{max} *= 250 bytes*, and a framing header of H_{size} *= 4 bytes* (leaving a 246 bytes payload slice), the stream chunks into approximately 196 sequential radio blocks (*48,000/246 = 196.14 ≈ 196 packets*).

Transmission Performance & Voice Feasibility

Voice testing will evaluate sustained payload throughput versus encoded audio rate, end-to-end delay, and qualitative speech intelligibility. Computational criteria for tracking raw physical layer timing follow the metrics established in this reference site [16] for LoRa packet time-on-air and total message delivery time estimation.

LoRa Packet Time-on-Air (T_{packet})

The estimated airtime duration for an individual frame symbol burst, derived from the Semtech SX1262 modem modulation formula [17]:

$$T_{\text{packet}} = T_{\text{sym}} \times \left(N_{\text{preamble}} + 4.25\right) + T_{\text{sym}} \times \max\left(\left\lceil \frac{8\text{PL} - 4\text{SF} + 28 + 16\text{CRC} - 20\text{H}}{4(\text{SF} - 2\text{DE})} \right\rceil \times (\text{CR} + 4), 0\right)$$

Where $Tsym = \frac{2^{SF}}{BW}$ is symbol duration, *PL* is packet payload size (246 bytes), *SF* is Spreading Factor, *BW* is Bandwidth, *CRC* is cyclic redundancy check state, *H* is header presence, *DE* is low data rate optimization, and *CR* is coding rate.

Total Message Delivery Time Estimate (T_{total})

The expected end-to-end transaction latency required to stream a whole 3-second recording:

$$T_{\text{total}} = N_{\text{chunks}} \times \left(T_{\text{packet}} + \Delta_{\text{pace}}\right) + T_{\text{processing}}$$

Where Δ_{pace} is the pacing/guard delay injected between SPI commands, and $T_{processing}$ is local buffer compilation and encryption latency.

7.2 Experiment Matrix

The planned experiments include:

- Varying spreading factor, bandwidth, coding rate, and transmit power
- Enabling and disabling encryption
- Injecting altered or repeated packets
- Varying message size and fragmentation settings
- Testing acknowledgment and retry strategies
- Comparing known data payloads versus compressed voice payloads

Each run will record node identifiers, parameter settings, timestamps, sequence numbers, and observed errors to ensure reproducible and comparable results.

8. References and Existing Work

8.1 References

[1] Meshtastic, “Audio Module Configuration.” https://meshtastic.org/docs/configuration/module/audio/

[2] Whitman B., “loracamera.” https://github.com/bwhitman/loracamera

[3] Espressif Systems, “ESP-IDF Programming Guide.” https://docs.espressif.com/projects/esp-idf/en/stable/esp32/index.html

[4] Heltec Automation, “WiFi LoRa 32 V4.” https://heltec.org/project/wifi-lora-32-v4/

[5] Espressif Systems, “ESP32.” https://www.espressif.com/en/products/socs/esp32

[6] Semtech, “SX1261/SX1262 Datasheet,” Apr. 2025. https://www.semtech.com/products/wireless-rf/lora-connect/sx1262

[7] A. Augustin, J. Yi, T. Clausen, and W. M. Townsley, “A Study of LoRa: Long Range & Low Power Networks for the Internet of Things,” Sensors, vol. 16, no. 9, Art. 1466, 2016. https://doi.org/10.3390/s16091466

[8] F. Adelantado, X. Vilajosana, P. Tuset-Peiro, B. Martinez, J. Melià-Seguí, and T. Watteyne, “Understanding the Limits of LoRaWAN,” IEEE Communications Magazine, 2017. https://arxiv.org/abs/1607.08011

[9] X. Yang, E. Karampatzakis, C. Doerr, and F. Kuipers, “Security Vulnerabilities in LoRaWAN,” in Proc. IEEE/ACM Third Int. Conf. on Internet-of-Things Design and Implementation (IoTDI), 2018. https://pure.tudelft.nl/ws/files/46032668/IoTDI2018.pdf

[10] M. Dworkin, “Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC,” NIST SP 800-38D, Nov. 2007. https://doi.org/10.6028/NIST.SP.800-38D

[11] S. Kent, “IP Encapsulating Security Payload (ESP),” RFC 4303, Dec. 2005. https://www.rfc-editor.org/info/rfc4303

[12] A. Minaburo, L. Toutain, C. Gomez, D. Barthel, and J.-C. Zúñiga, “SCHC: Generic Framework for Static Context Header Compression and Fragmentation,” RFC 8724, Apr. 2020. https://www.rfc-editor.org/info/rfc8724

[13] D. Rowe, “Codec 2,” open-source speech codec. https://github.com/drowe67/codec2

[14] G. Almes, S. Kalidindi, and M. Zekauskas, “A Round-trip Delay Metric for IPPM,” RFC 2681, Sept. 1999. https://www.rfc-editor.org/info/rfc2681

[15] G. Almes, S. Kalidindi, M. Zekauskas, and A. Morton, Ed., “A One-Way Loss Metric for IP Performance Metrics (IPPM),” RFC 7680, Jan. 2016. https://www.rfc-editor.org/info/rfc7680

[16] RF Wireless Expert Team, “LoRaWAN Airtime Calculator and LoRa Packet Duration Calculation,” RF Wireless World, Aug. 25, 2026. Available: https://www.rfwireless-world.com/calculators/lorawan-airtime-calculator

[17] Semtech Corporation, "SX1272/3/6/7/8 LoRa Modem Designer's Guide," Application Note AN1200.13, Jul. 2013. [Online]. Available: https://www.mouser.com/pdfdocs/semtech-lora-modem-design.pdf

8.2 Existing Work to Reuse

The project will reuse or draw design guidance from:

- Vendor SDK examples for serial, LoRa, storage, and I2S
- Authenticated encryption reference implementations compatible with ESP32
- LoRa application examples involving message framing and telemetry as seen from the following work on LoRa camera and telemetry project example [12]
- Meshtastic and similar open-source LoRa communication projects for general architectural reference only, not as a complete drop-in solution
- All reused code, libraries, and design references will be explicitly cited in the final document.

9. Planned Final-Document Additions

After the CDR submission, this document would be updated into the final project document with the following additions:

- Assembly and wiring instructions
- Development-environment setup procedure
- Firmware builds and flash instructions
- Host testbed setup and execution steps
- Screenshots of serial logs, host UI or scripts, and analysis plots
- Measured packet delivery, throughput, latency, and security test data
- Photographs of the hardware setup
- Known issues and recommended future improvements

The final version will provide enough detail that another student could rebuild, modify, and extend the system without direct assistance from the original team.